

MTSC2025

中国互联网测试开发大会

TESTING SUMMIT CONFERENCE CHINA 2025

上海
站

质效革新·智领未来

2025/7/12 | 上海喜来登由由大酒店 主办方: TesterHome

分享人：郭同闯

5年Android客户端性能优化研发经验

目前主要负责鸿蒙性能检测工具开发、
性能防劣化建设



MTSC2025 中国互联网测试开发大会 上海站
TESTING SUMMIT CONFERENCE CHINA 2025

抖音鸿蒙客户端性能防劣化建设

质效革新·智领未来

主办方: TesterHome

目 录

CONTENTS

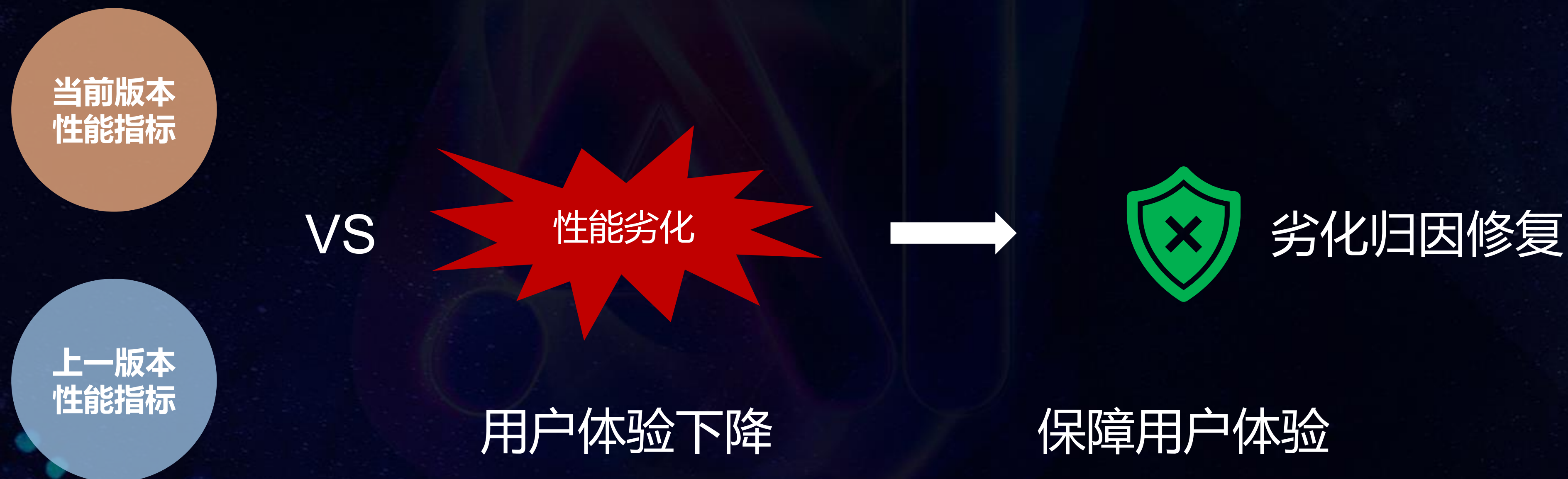
- 背景介绍
- 基于Trace Diff的防劣化方案
- 针对多层Import的防劣化方案
- 总结与展望

一、背景介绍

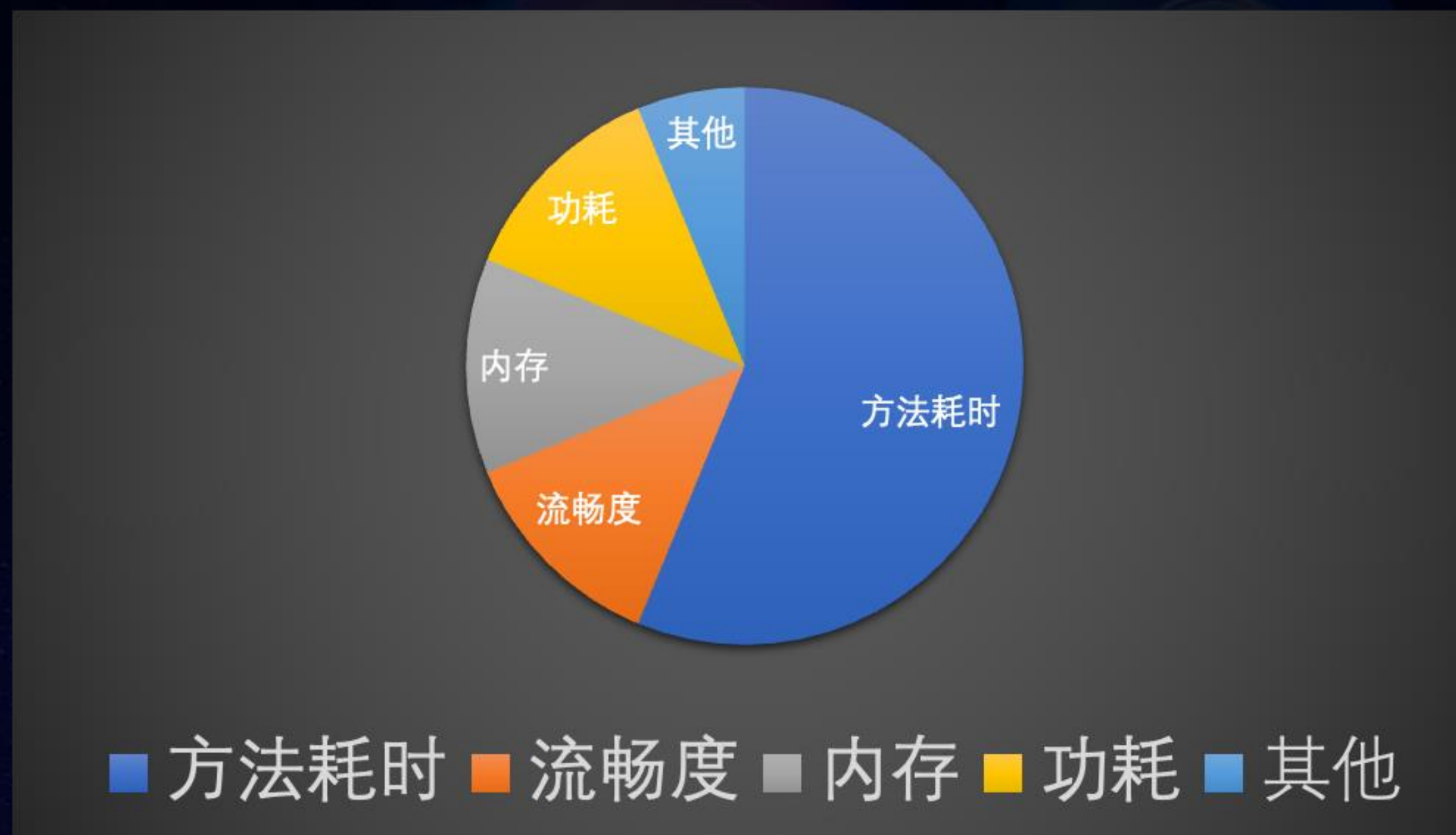
鸿蒙发展历程



什么是性能防劣化



性能指标



优先重点建设方法耗时防劣化，逐步完善内存、流畅度等防劣化能力

传统性能防劣化

数据不准确

数据波动大误报率高

问题难定位

纯人工归因且周期长

拦截不及时

线上指标拦截太迟

劣化易逃逸

其他优化点掩盖劣化问题

精准性能防劣化



全链路防劣化



数据全面准确



归因快速精准

二、基于Trace Diff的防劣化方案

版本间防劣化



版本选择

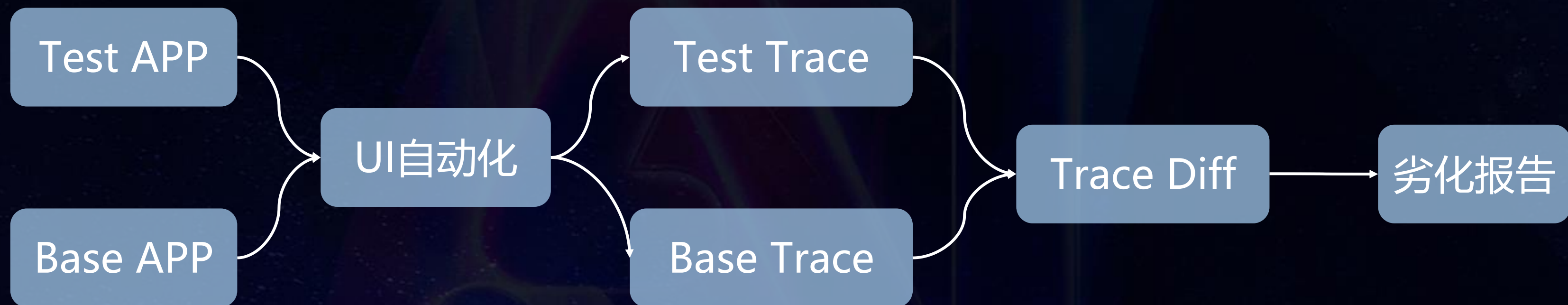
Webhook
CICD事件

发版事件

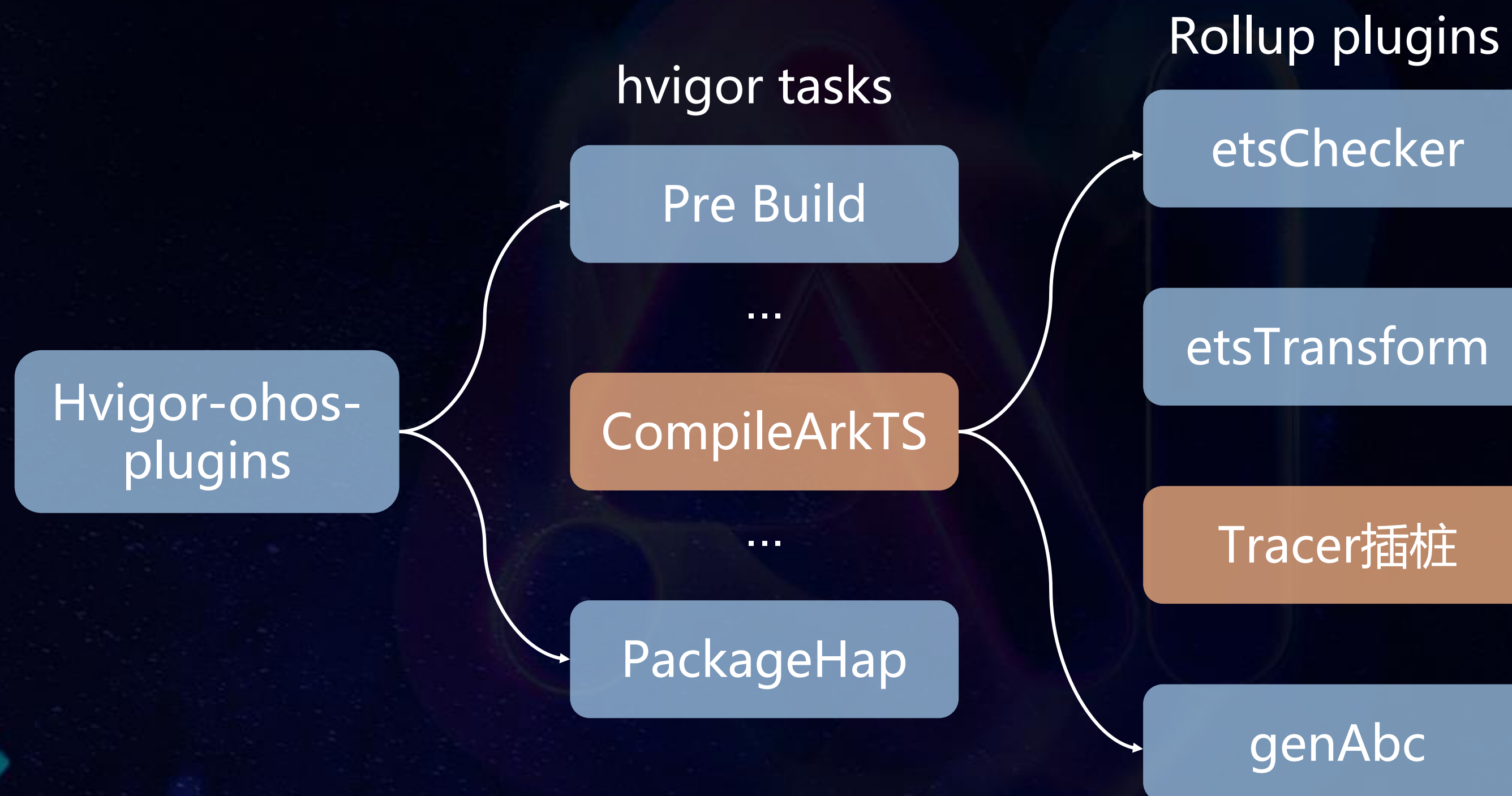
测试包：当前发版节点的commit id

基准包：上一正式版本的commit id

技术方案



Trace插桩工具



Trace插桩工具

插桩目标:

- 01 Constructor: 类的构造方法
- 02 MethodDeclaration: 类方法
- 03 FunctionDeclaration: 函数

插桩内容:

- 01 hiTraceMeter.startTrace() 开始trace打标
- 02 hiTraceMeter.finishTrace() 结束trace打标

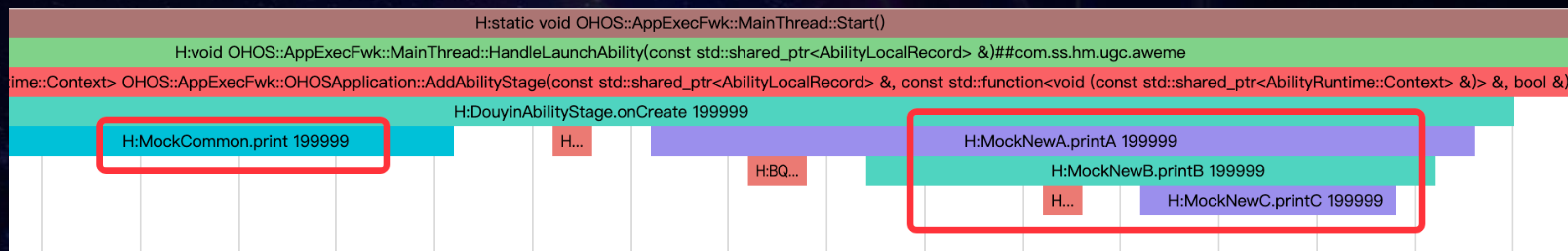
Trace插桩工具

AST 插桩示例

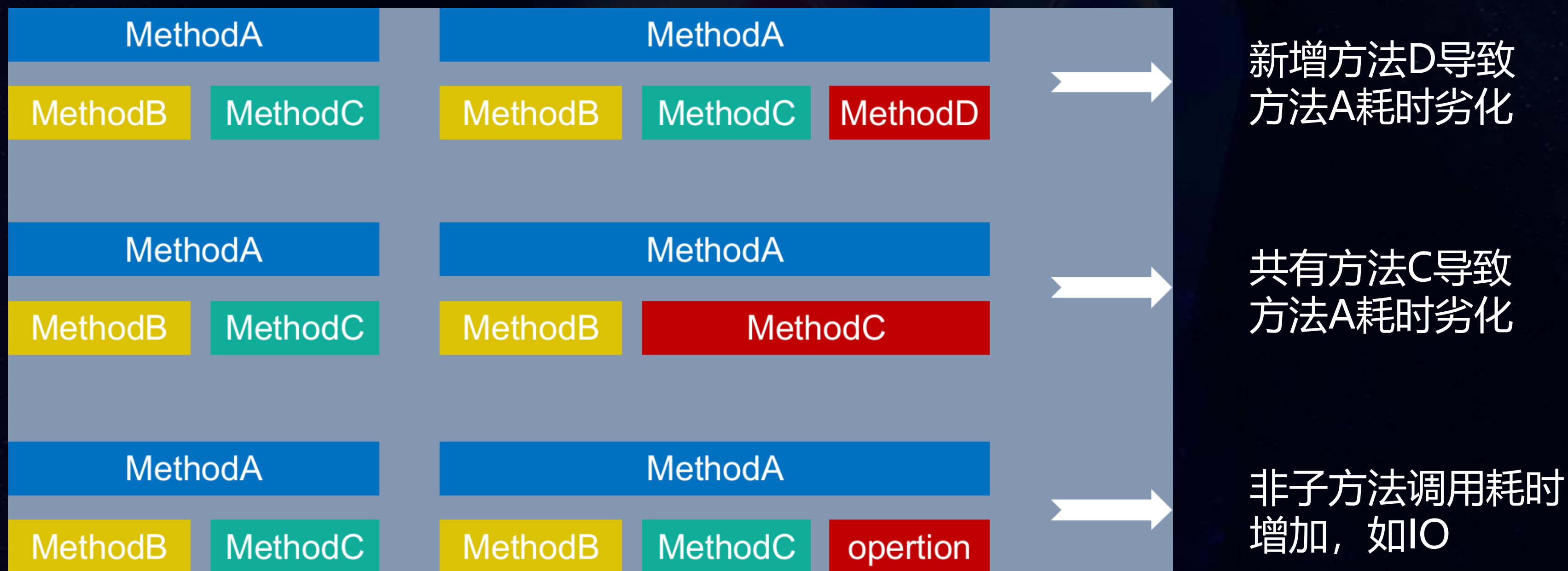
```
function printC(){
  hiTraceMeter.startTrace("MockNewC.print", 199999)
  console.log("AST test")
  hiTraceMeter.finishTrace("MockNewC.print", 199999)
}
```

```
factory.createExpressionStatement(factory.createCallExpression(
  factory.createPropertyAccessExpression(
    factory.createIdentifier("hiTraceMeter"),
    factory.createIdentifier("startTrace")
  ),
  undefined,
  [
    factory.createStringLiteral("MockNewC.print"),
    factory.createNumericLiteral("199999")
  ]
)),
```

插桩后trace示例



Trace Diff分析



准确率优化——减小波动

维度	类型	目标
设备环境	网络环境	减少网络波动(网络类型、网速等)导致的误报
	进程清理	清理不必要进程，避免资源干扰
软件环境	缓存清理、账号登录	使用相同的用户状态和权限进行测试

准确率优化——减小波动

多次测试分析命中率

第一次测试



第二次测试



...

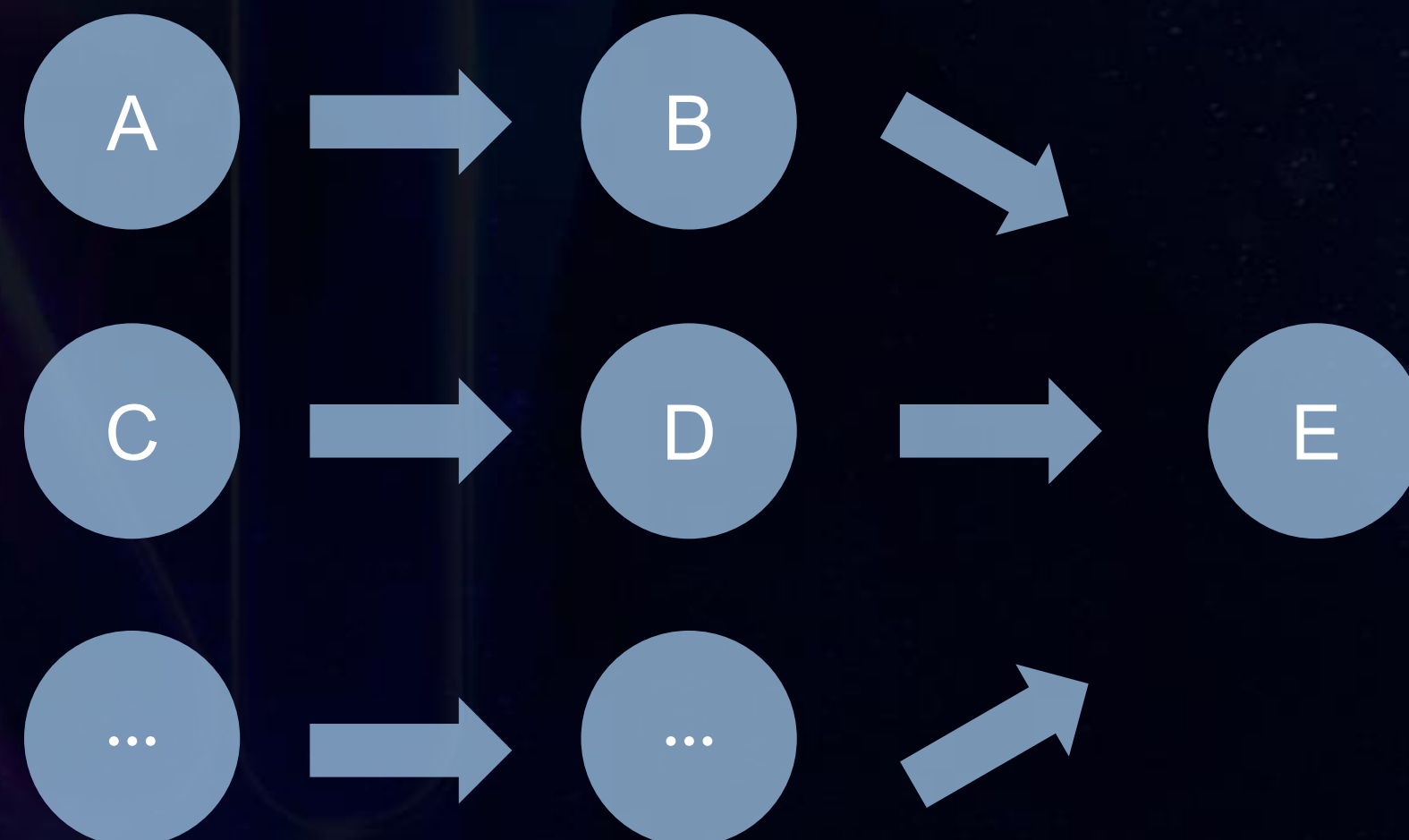


第八次测试



命中率低于60%无效

静态分析调用链



调用链未变动无效

准确率优化——网络一致性

01

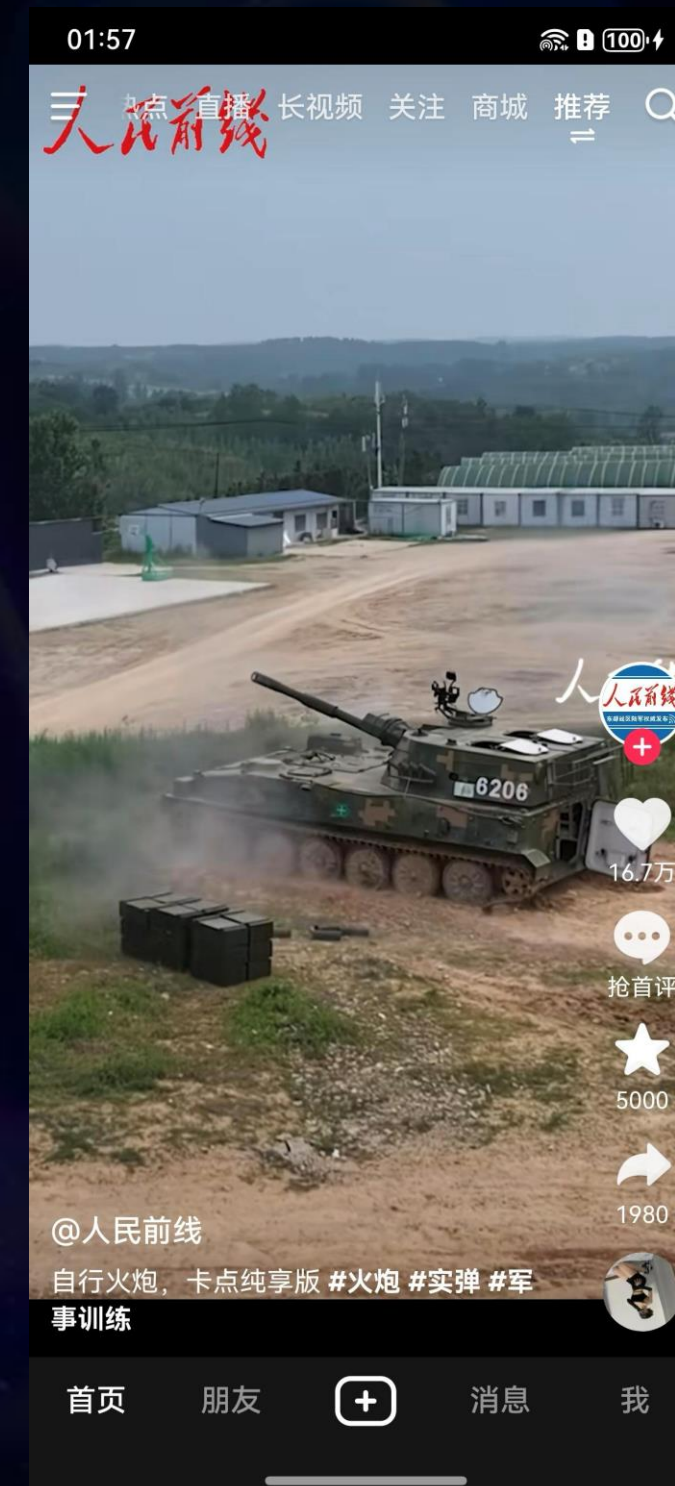
Feed视频内容不同，导致下载、视频解码耗时不同

02

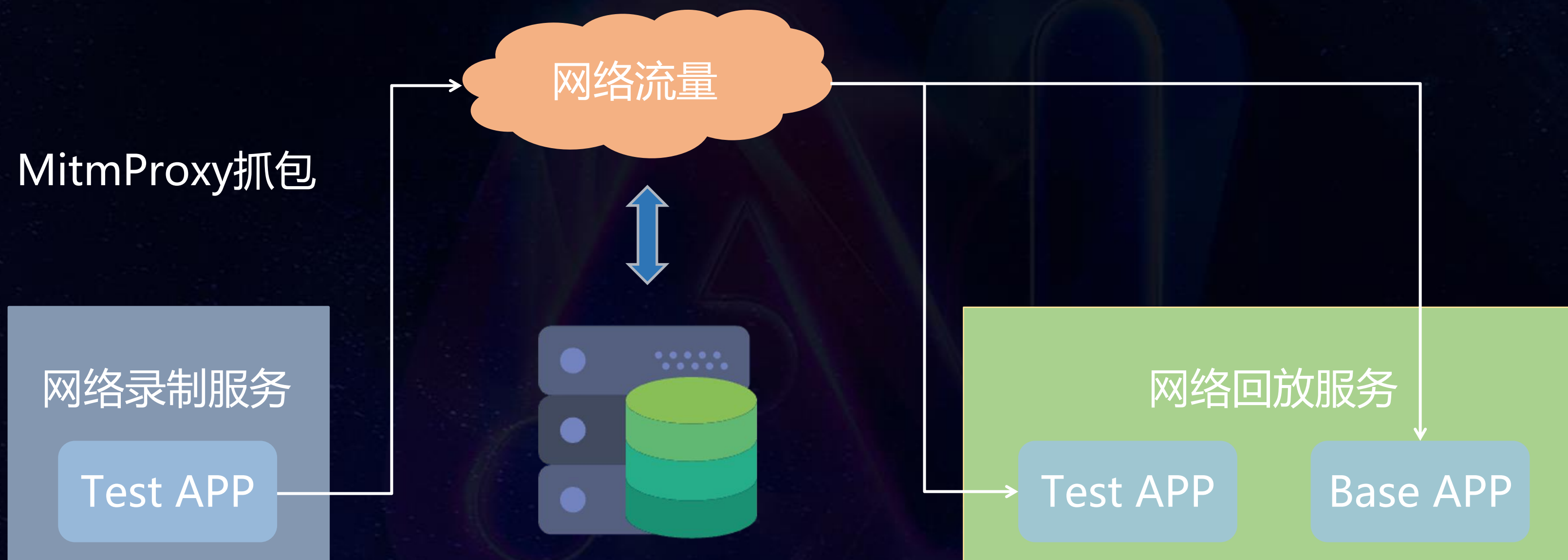
AB实验等功能受后端下发配置影响，会执行不同的逻辑分支

03

某些请求在不同的响应时延下，会执行重试、兜底等策略



准确率优化——网络一致性



对网络数据进行录制回放，保障双包对比测试中网络一致性

准确率优化——堆栈相似度

方法重命名

```
function funA( ){  
    ...  
}
```



```
function funB( ){  
    ...  
}
```

方法封装

```
function funA( ){  
    ...  
}
```



```
function funB( ){  
    funA( )  
    ...  
}
```

方法迁移

```
function funB( ){  
    funA( )  
    ...  
}
```



```
function funC( ){  
    funA( )  
    ...  
}
```

准确率优化——堆栈相似度

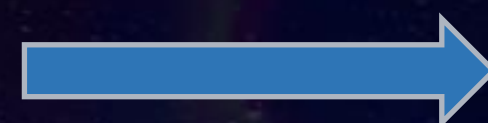
fun A删除节点

C	D
H	K

fun B新增节点

C	E
H	D
	K

相似度匹配

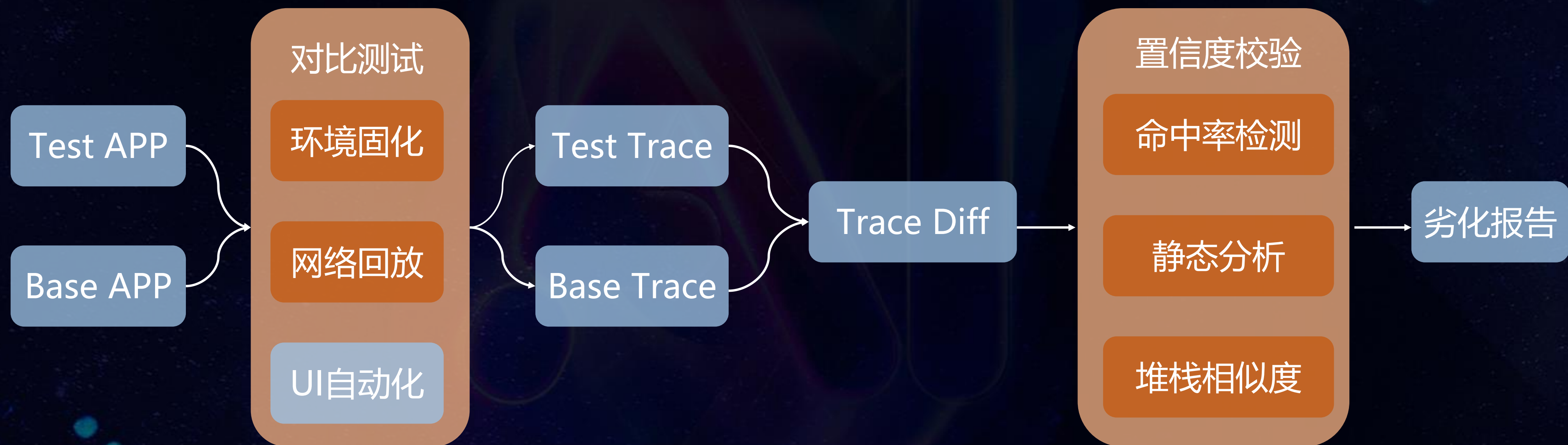


A - C - H
B - C - H

A - D - K
B - E - D - K

相似堆栈的节点为 $x_1 \sim x_i$,
每个节点的权重为 $y_1 \sim y_i$, 堆栈相似
度为 $z_1 \sim z_i$
堆栈相似度 = $y_1 * z_1 + \dots + y_i * z_i$

优化后方案



效果展示

```
computeTask10(){
  let count = 0;
  while (count < 1260000) {
    count++;
  }
}

computeTask30(){
  let count = 0;
  while (count < 3780000) {
    count++;
  }
}
```

```
aboutToAppear() {
  this.computeTask5()
  this.computeTask10()
  this.computeTask30()
  this.computeTask50()
  this.computeTask100()
  this.computeTask200()

  // let task = new taskpool.Task(computeTaskAsync)
  // taskpool.execute(task)
```

Mock 5 ~ 200ms 多组方法耗时劣化，以观测劣化归因效果

效果展示



准确率由 40% 提升至 80%

诊断结果

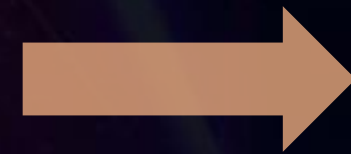
判断结果 劣化

劣化点名称	类型 ▾	线程	阶段	置信度	命中率 ⓘ ▾	劣化原因 关键线程 后台线程
H:HomePageNode.computeTask100	新增方法	ss.hm.ugc.aweme/...	start	1	100%	CPU Duration = 104.6 ms Wall Duration = 104.8 ms
H:HomePageNode.computeTask200	新增方法	ss.hm.ugc.aweme/...	start	1	100%	CPU Duration = 207.1 ms Wall Duration = 207.3 ms
H:HomePageNode.computeTask5	新增方法	ss.hm.ugc.aweme/...	start	1	100%	CPU Duration = 5.8 ms Wall Duration = 5.8 ms
H:HomePageNode.computeTask10	新增方法	ss.hm.ugc.aweme/...	start	1	100%	CPU Duration = 10.4 ms Wall Duration = 10.4 ms
H:HomePageNode.computeTask30	新增方法	ss.hm.ugc.aweme/...	start	1	100%	CPU Duration = 31.3 ms Wall Duration = 31.4 ms
H:HomePageNode.computeTask50	新增方法	ss.hm.ugc.aweme/...	start	1	100%	CPU Duration = 52.1 ms Wall Duration = 52.2 ms

MR防劣化



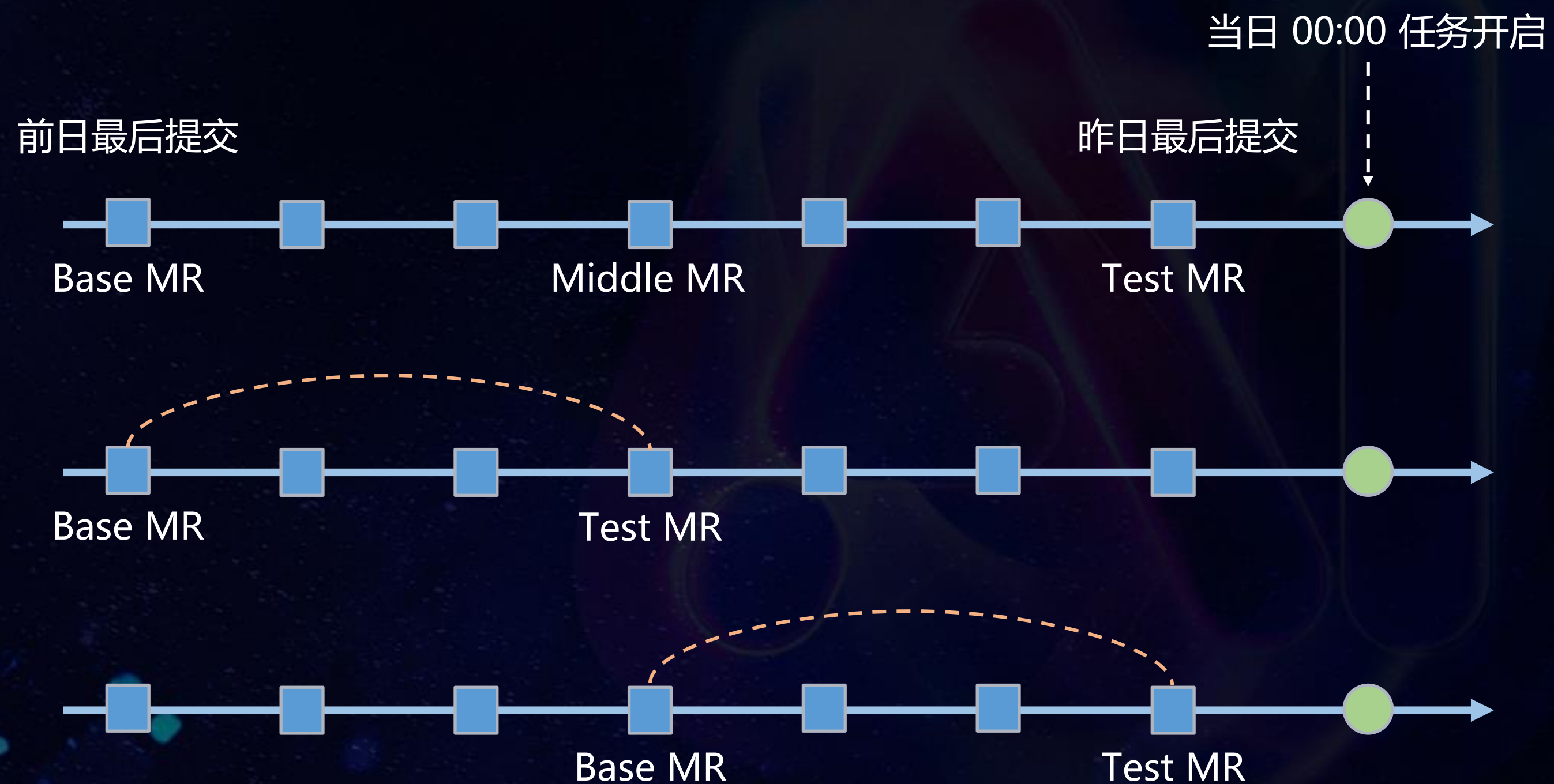
- ◆ 劣化修复量大，无法跟版修复
- ◆ 灰度阶段发现劣化略晚，开发阶段发现更佳



MR防劣化

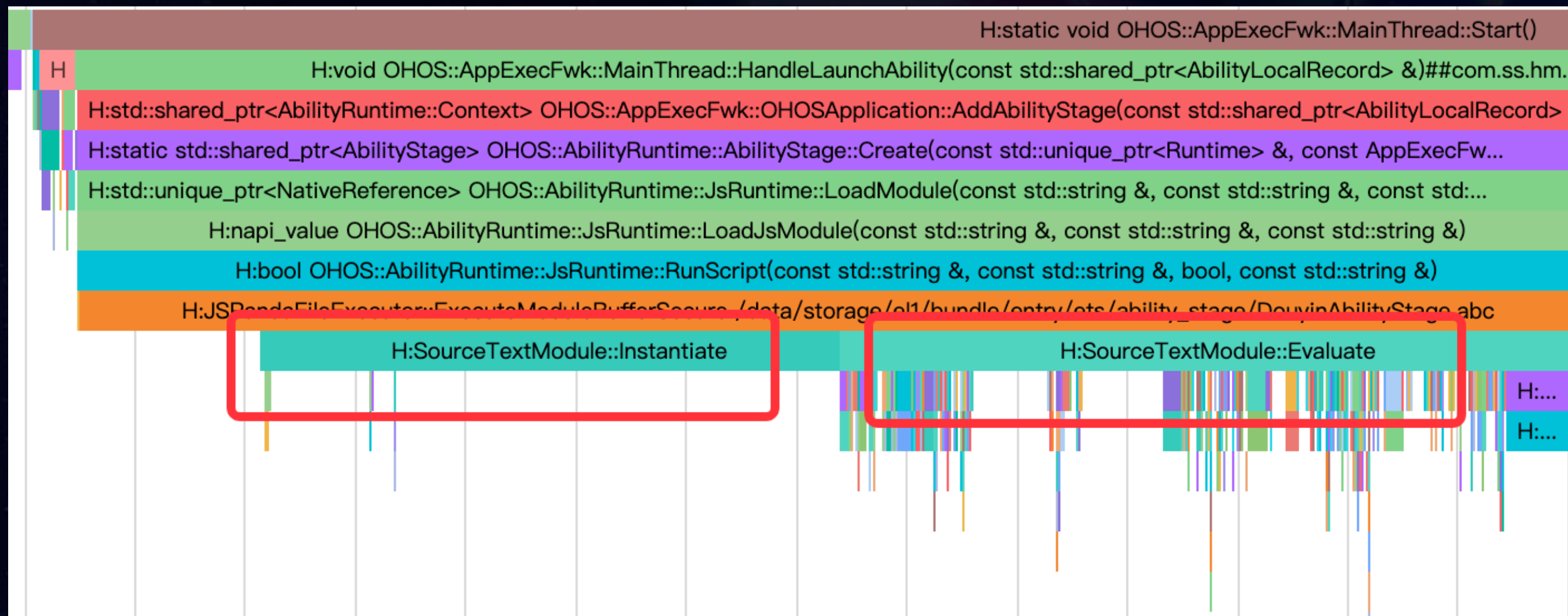


MR防劣化



三、针对多层Import的防劣化方案

Import流程分析



步骤1: Instantiate
构建Import 依赖树

步骤2: Evaluate
执行文件初始化

Import流程——Instantiate

- Local Import

```
import {classA} from './classA'
```



- Indirect Import

```
import {classA} from '@douyin/test/Index'
```

```
export {classA} from './classA'
```

```
export * from './classA'
```



Import流程——Evaluate

全局变量初始化

```
let a = new TestA( )
```

```
export const b =  
new TestB( )
```

顶层函数执行

```
function funA( ){
```

```
  ...
```

```
}
```

```
funA( )
```

so的dlopen

```
import {nativeA}  
from 'libtest.so'
```

为什么做Import防劣化

01

Trace 插桩无法全量覆盖，如箭头函数，Evaluate 函数下存在较大函数空白，无法定位劣化函数

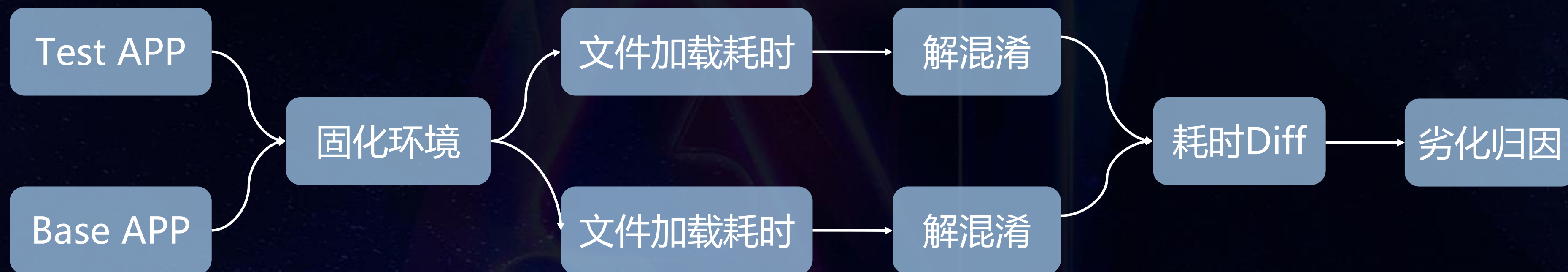
02

Evaluate 函数下是顺序执行各个类初始化函数，没有父子关系，无法确定上层引入源，即无法归因

03

全量Lazy Import 会增加运行时初始化耗时，可能会造成任务阻塞

技术方案



文件加载耗时采集

```
parentModule 3: com.huawei.hms.mapservice.kit/mapLibrary@OHMapBaseSDK/ets/map/cache/ViewDataCache, cost time: 0.073ms
unused file 815: com.huawei.hms.mapservice.kit/mapLibrary@OHMapBaseSDK/ets/map/cache/ViewDataCache, cost time: 0.073ms
parentModule 1: com.huawei.hms.mapservice.kit/mapLibrary@OHMapBaseSDK/Index
parentModule 2: com.huawei.hms.mapservice.kit/mapLibrary@OHMapBaseSDK/ets/map/MapView
parentModule 3: com.huawei.hms.mapservice.kit/mapLibrary@OHMapBaseSDK/ets/map/UiSettings
parentModule 4: com.huawei.hms.mapservice.kit/mapLibrary@OHMapBaseSDK/ets/map/component/CompassView
parentModule 5: com.huawei.hms.mapservice.kit/mapLibrary@OHMapBaseSDK/ets/map/component/InfoWindowView
parentModule 6: com.huawei.hms.mapservice.kit/mapLibrary@OHMapBaseSDK/ets/map/component/MyLocationView
parentModule 7: com.huawei.hms.mapservice.kit/mapLibrary@OHMapBaseSDK/ets/map/component/ScaleView
parentModule 8: com.huawei.hms.mapservice.kit/mapLibrary@OHMapBaseSDK/ets/map/component/ZoomView
parentModule 9: com.huawei.hms.mapservice.kit/mapLibrary@OHMapBaseSDK/ets/map/impl/IMapImpl
未使用文件 parentModule 10: com.huawei.hms.mapservice.kit/mapLibrary@OHMapBaseSDK/ets/map/impl/MarkerImpl
parentModule 11: com.huawei.hms.mapservice.kit/mapLibrary@OHMapBaseSDK/ets/map/listener/MapPicker
unused file 816: com.huawei.hms.mapservice.kit/mapLibrary@OHMapBaseSDK/ets/map/UiSettings, cost time: 0.079ms 文件加载耗时
parentModule 1: com.huawei.hms.mapservice.kit/mapLibrary@OHMapBaseSDK/ets/map/impl/IMapImpl
unused file 817: com.huawei.hms.mapservice.kit/mapLibrary@OHMapBaseSDK/ets/map/model/AnimationRepeatMode, cost time: 0.008ms
parentModule 1: com.huawei.hms.mapservice.kit/mapLibrary@OHMapBaseSDK/ets/map/model/animation/Animation
parentModule 2: com.huawei.hms.mapservice.kit/mapLibrary@OHMapBaseSDK/ets/map/utils/RepeatModeUtil
```

1. hdc shell param set persist.ark.properties 0x200105c

2. hdc file recv data/app/el2/100/base/\${bundleName}/files/\${bundleName}_redundant_file.txt ./

构建父子关系

Parents

file A

file B

file C

file X

file Y

Children

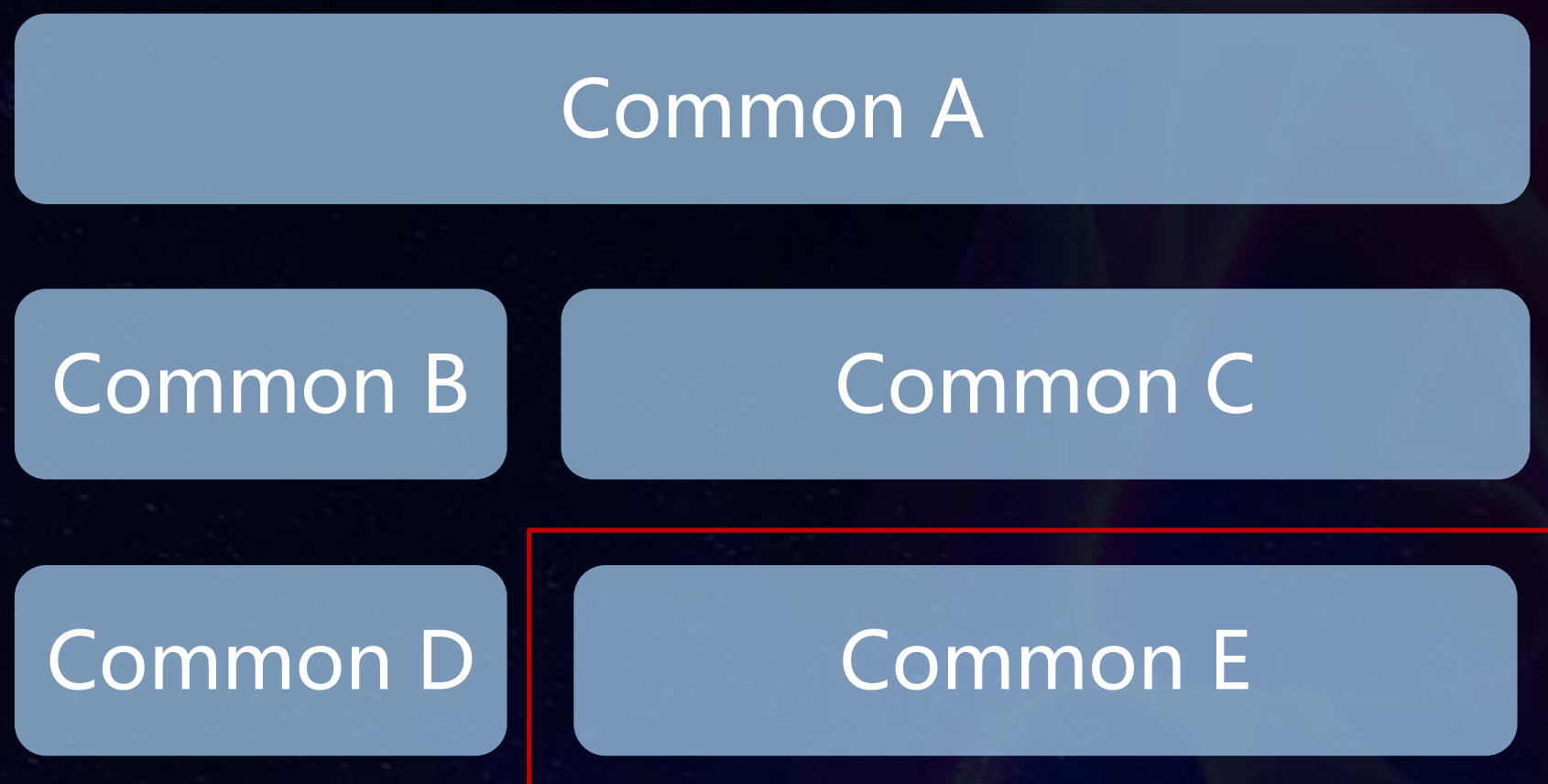
file D

file E

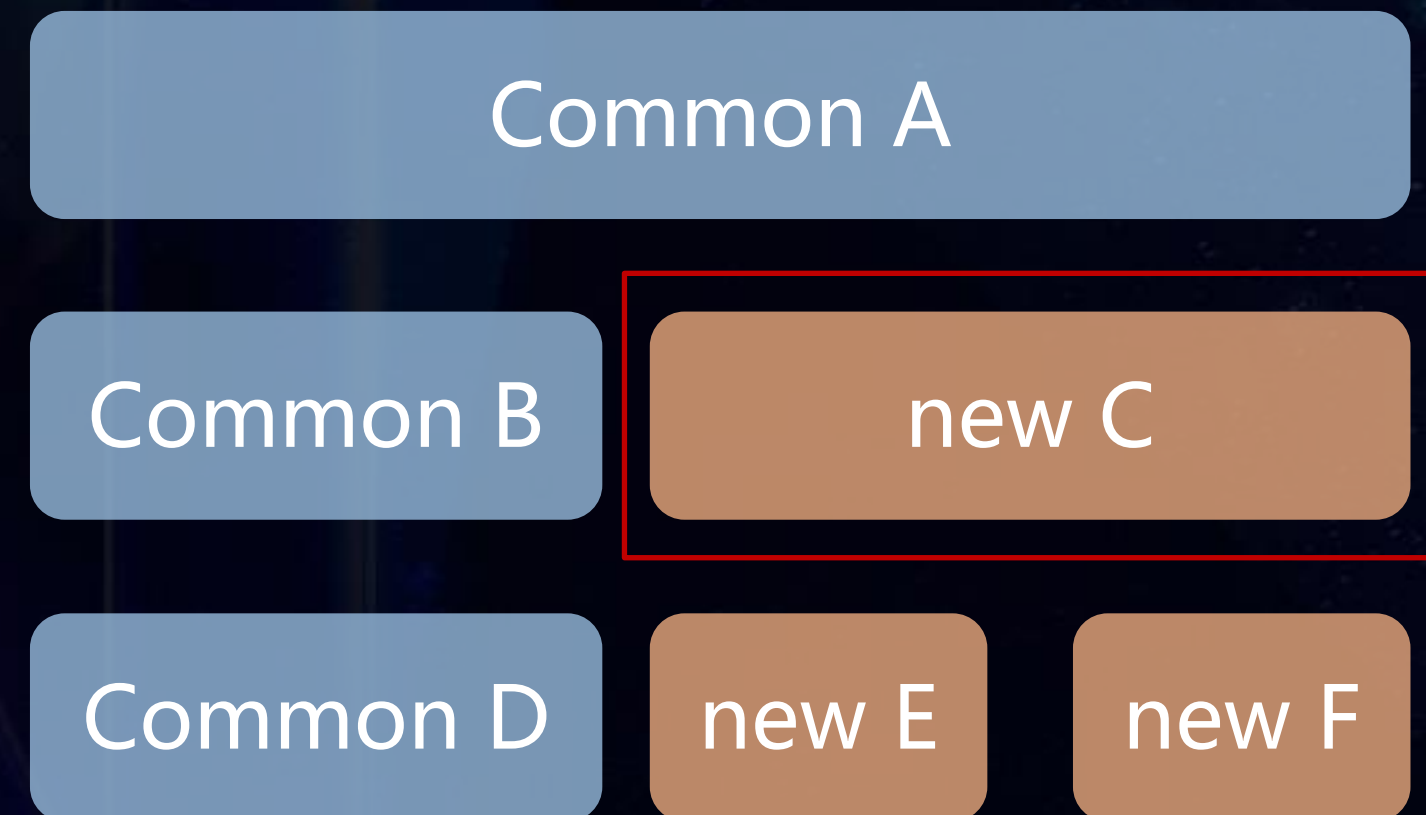
file F

D对X 唯一影响

劣化归因



共有劣化：定位至最下层劣化节点



新增劣化：定位至最上层劣化节点

效果展示

```
MockNewA.ets × MockNewB.ets × MockNewC.ets × MockCommon.ets ×
1  const mockDelay = () => {
2    console.log("exec mock new mockDelay")
3    let count = 0;
4    while (count < 650000) {
5      count++;
6    }
7  };
8  mockDelay();
9
10 export function printC(){
11   console.log("exec mock new print C")
12 }
```

新增劣化

```
MockNewA.ets × MockNewB.ets × MockNewC.ets × MockCommon.ets ×
1  const mockDelay = () => {
2    console.log("exec mock common mockDelay")
3    let count = 0;
4    while (count < 1910000) {
5      count++;
6    }
7  };
8  mockDelay();
9
10 export class MockCommon{
11   print(){
12     console.log("exec mock common print")
13   }
14 }
```

共有劣化

诊断结果

判断结果 劣化

劣化文件	类型	使用状态	阶段	置信度	命中率	劣化耗时
ability_stage/MockNewA	新增劣化	used	start	1	100%	Wall Duration Diff = 5.4 ms
ability_stage/MockCommon	共有劣化	used	start	1	100%	Wall Duration Diff = 5.1 ms

四、总结与展望

业务落地总结

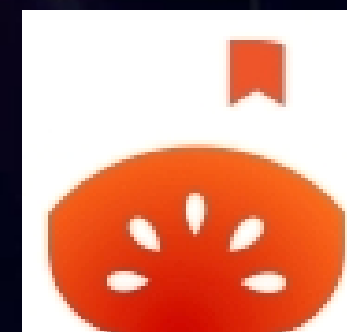
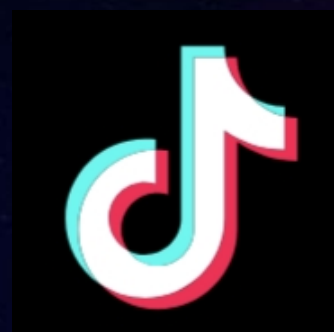
覆盖场景

新老用户启动

流畅度

业务收益：抖音25年H1拦截
有效函数劣化共计 **1400+ms**

接入应用



技术指标：全链路召回率
90+%，准确率 80%

存在的问题

消费效率低

缺少静态根因分析

仍不够前置

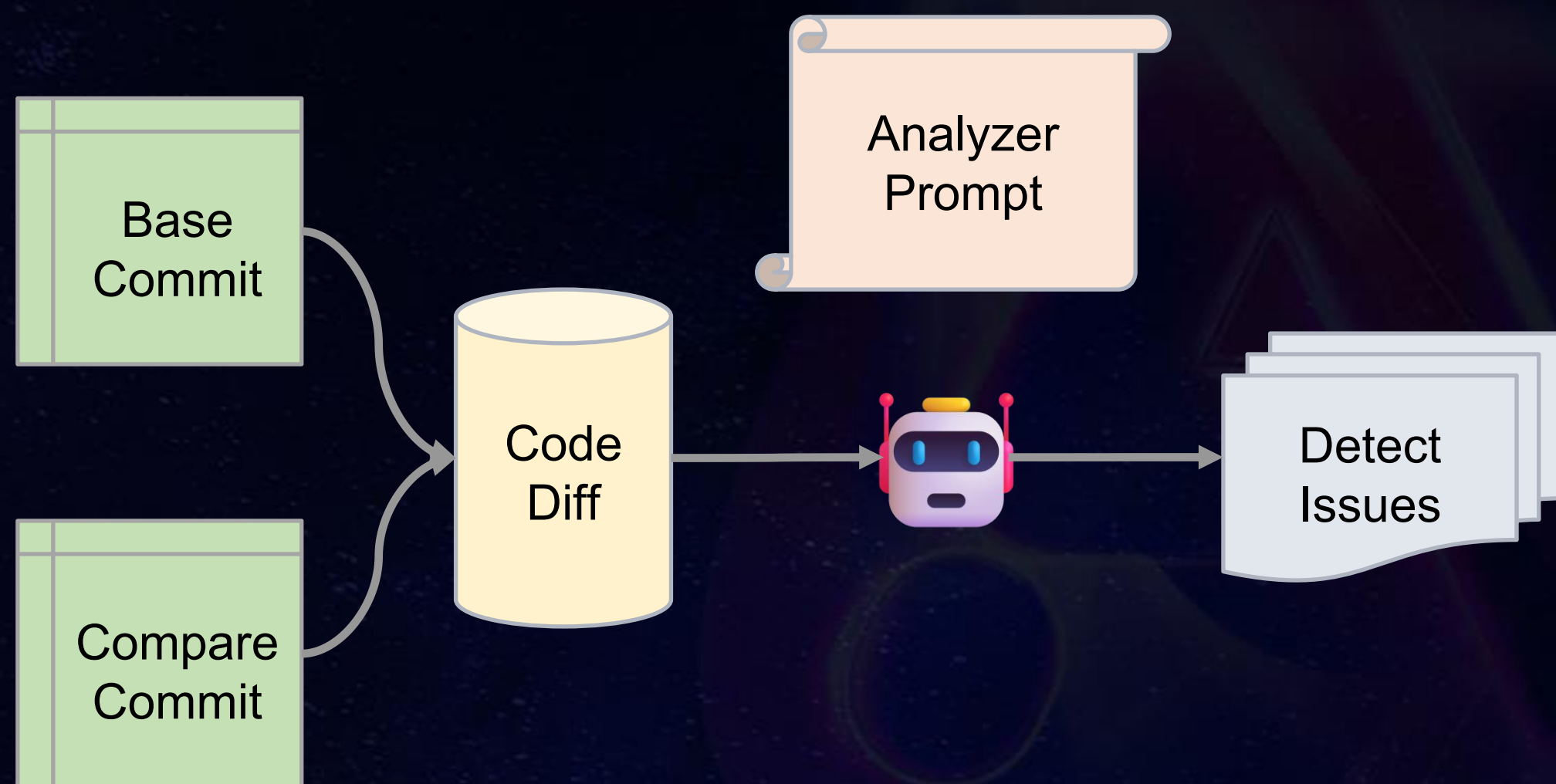
期望对每个MR做劣化监控

执行成本高

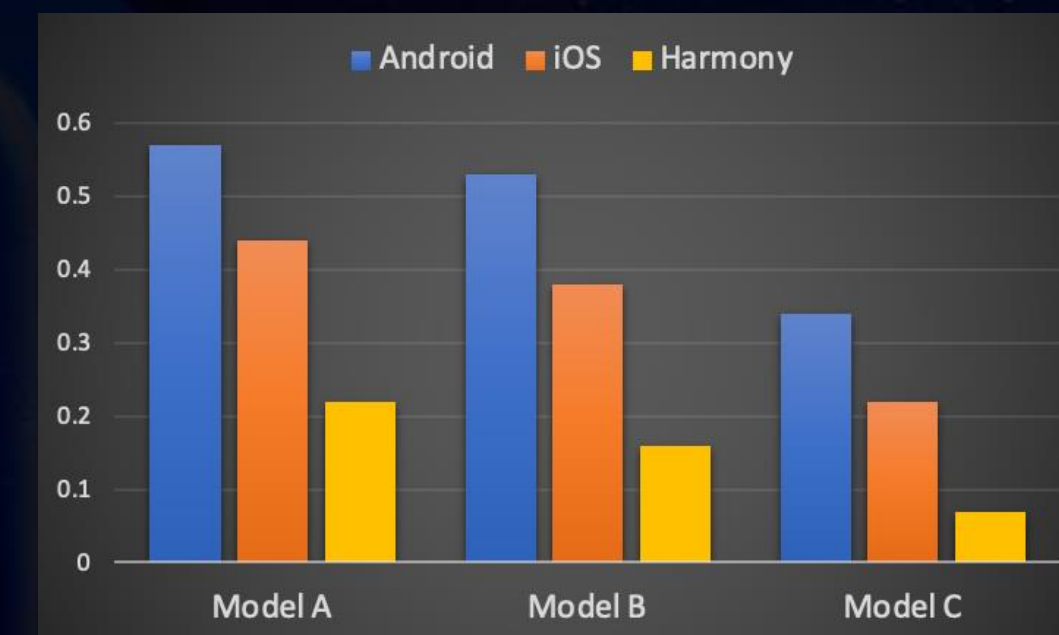
高峰期依赖大量设备

大模型赋能实践和展望

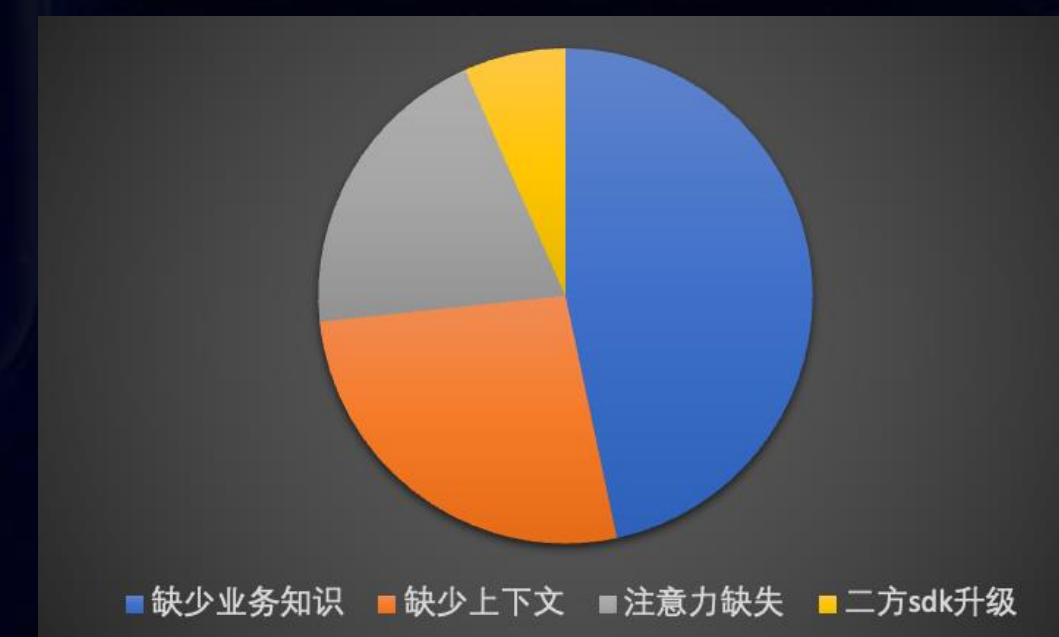
基于大模型的劣化智能识别



基于CoT的劣化发现CodeAgent

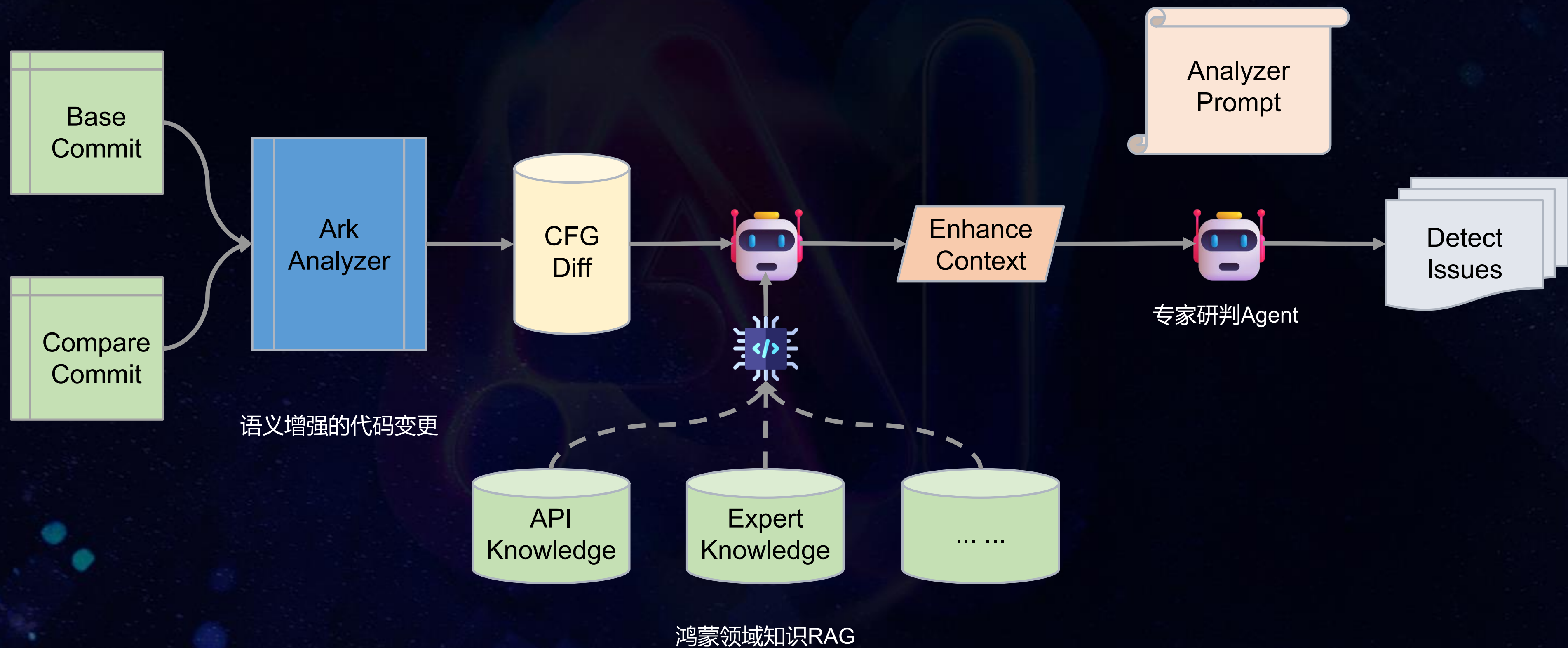


大模型多端劣化识别准确率



Harmony劣化识别失败原因

多智能体优化方案



大模型赋能防劣化服务

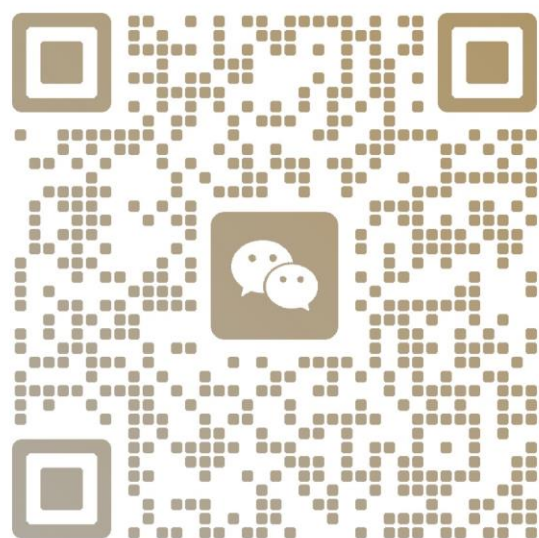


谢谢

THANKS



Oops
河北 邢台



扫一扫上面的二维码图案，加我为朋友。